LLM Red-Team Agents vs. Real Honeypot Traffic

Project Documentation — architecture, setup, usage, data formats and script reference

Keshav, Anshul Sharma, Parth Kansal, Virain Nayar

Chandigarh University

October 2026

CONTENTS

1. Project Overview 1

1.1 What this project does 1

1.2 Short answer 1

1.3 Key numbers 2

1.4 Who should read what 2

1.5 Safety and ethics 2

2. System Architecture 3

2.1 The three blocks 3

2.2 Data flow 4

2.3 Components 6

2.4 Deployment 7

2.5 Repository layout 8

3. Installation and Setup 9

3.1 Requirements 9

3.2 Steps 9

3.3 The real dataset 9

3.4 API keys 9

3.5 Checking the installation 10

4. Running the Pipeline 11

4.1 Overview 11

4.2 Optional: the live honeypot 11

4.3 Step 1: parse the real data 11

4.4 Step 2: label-free comparison 12

4.5 Steps 3 and 4: annotation 12

4.6 Step 5: transfer experiment 12

4.7 Steps 6 to 8: tables, figures, report 12

5. Data Formats 13
5.1 Overview 13
5.2 Raw synthetic session log (`logs/**/<session_id>.json`) 14
5.3 Normalised session (`normalized/*.jsonl`, one JSON object per line) 14
5.4 Annotation files (`annotation/`) 15
5.5 Result files (`results/`) 15
6. Script Reference 16
6.1 Analysis scripts 16
6.2 Shared library: session_features.py 17
6.3 Testbed scripts 17
6.4 Report scripts (`report/src/`) 18
7. Results Summary 19
7.1 Label-free comparison 19
7.2 Labels 19
7.3 Transfer 19
7.4 Robustness 20
7.5 Other findings 20
8. Troubleshooting and FAQ 21
8.1 Common problems 21
8.2 Frequently asked questions 21

1. Project Overview

1.1 What this project does

This repository answers one question: **can sessions generated by LLM "red-team" agents against a honeypot we own be used to train a classifier for real honeypot traffic?**

It contains everything needed to answer that question from scratch:

- a testbed built on the Cowrie SSH honeypot, with a static shell backend and an LLM-written shell backend;
- a harness that lets an LLM issue one shell command per turn against our own honeypot and records every turn;
- a parser for the public CyberLab honeynet dataset (real attacker sessions);
- shared code that normalises commands from both sources in the same way;
- a label-free comparison of the two corpora;
- tools for blind annotation, agreement measurement and adjudication;
- a four-quadrant transfer experiment;
- scripts that regenerate every table and figure, the paper (Springer LNNS format) and the project report.

1.2 Short answer

No, not with a single "explore the system" persona. The synthetic and real sessions are almost perfectly separable (verb Jensen-Shannon divergence 0.700, classifier AUC 1.000). A classifier trained on synthetic sessions reaches a macro-F1 of 0.205 on real sessions, against 0.983 when trained on real sessions, and adding synthetic data to real data gives no gain (0.981). The agents explore files and hunt credentials; most real sessions on the sample day ran one cryptominer dropper.

1.3 Key numbers

Table 1.1: Key facts of the project.

Item	Value
Synthetic sessions	120 = 3 attackers × 2 backends × 20
Attacker models	DeepSeek-V3.2, Gemma-3-4B, GPT-OSS-120B
LLM honeypot backend	Qwen3-32B
Real dataset	CyberLab honeynet, 15 November 2019 (Zenodo 10.5281/zenodo.3687527)
Real sessions / interactive sessions	76,785 / 1,672
Real behaviour templates	22
Annotation agreement	Cohen's $\kappa = 0.898$ on 184 items
Transfer (synthetic → real)	macro-F1 0.205 (one per template: 0.071)
Real → real	macro-F1 0.983
Real + synthetic → real	macro-F1 0.981

1.4 Who should read what

Table 1.2: Reading guide.

If you want to…	Read
Understand how the pieces fit	Chapter 2 (Architecture)
Install and run the project	Chapters 3 and 4
Read or reuse the data files	Chapter 5 (Data formats)
Know what every script does and its options	Chapter 6 (Script reference)
See the results in one place	Chapter 7
Fix a problem	Chapter 8 (Troubleshooting and FAQ)

1.5 Safety and ethics

The attacker harness is a research tool for **our own** honeypot only. It connects to exactly one configured host, has no scanning or target-discovery code, and the honeypot holds no real data. The real dataset is public (CC BY 4.0) and uses hashed source identifiers; nothing in this project tries to re-identify anyone. API keys are read from environment variables and are never written to logs or to the repository.

2. System Architecture

2.1 The three blocks

The system has three blocks (**Figure 2.**1):

1. **Synthetic data generation** in our own testbed: an attacker harness, three attacker models behind one API, and Cowrie running either its built-in shell or an LLM-written shell.
2. **Data preparation**: a streaming parser for the CyberLab export and a shared normalisation library used for both corpora.
3. **Analysis and outputs**: the label-free comparison, the annotation tools, the transfer experiment, and the scripts that write tables, figures, the paper and the report.

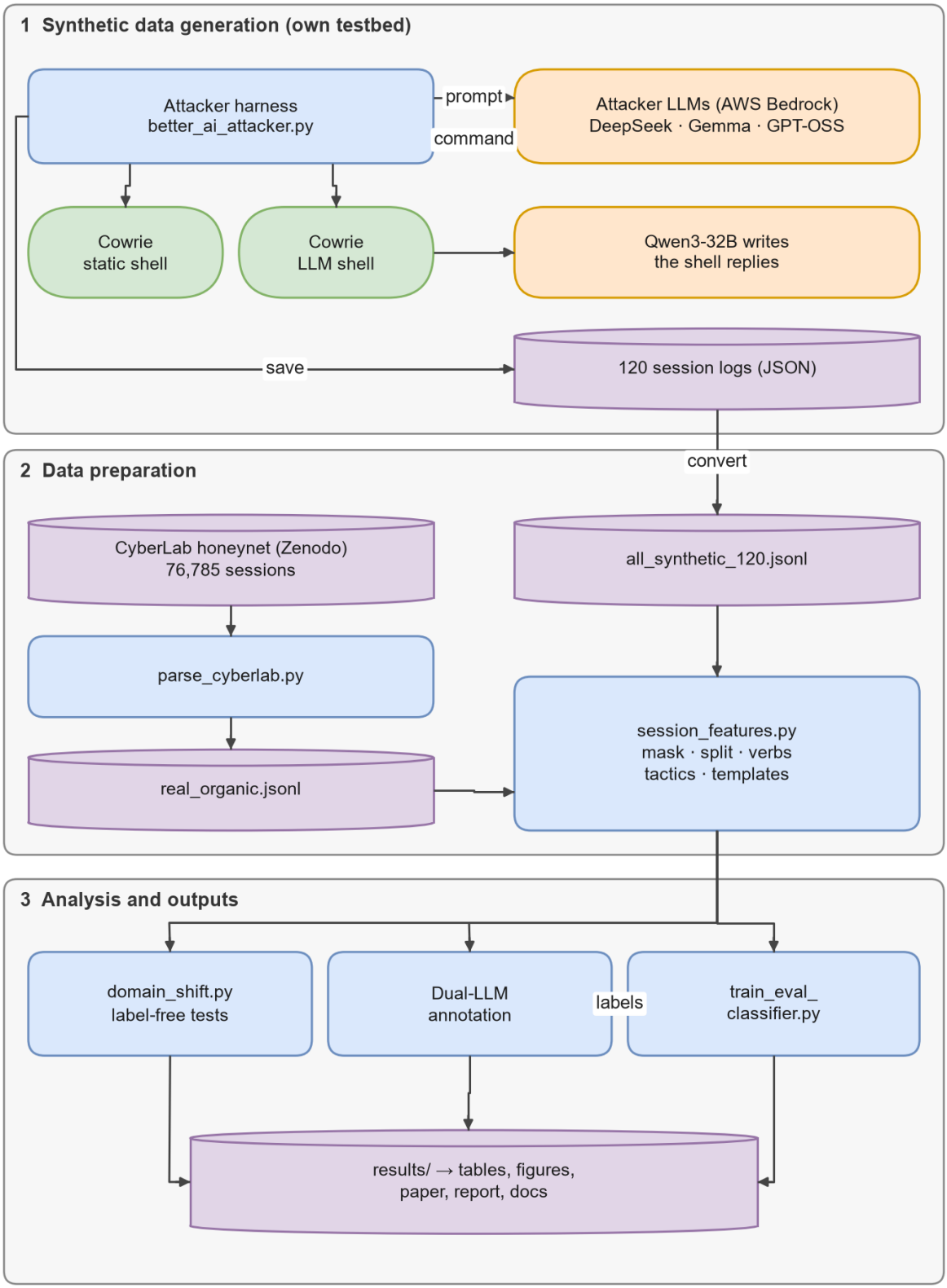


Figure 2.1: System architecture.

2.2 Data flow

Figure 2.2 shows how files move between processes. Each data store is a folder in the repository: `logs/`, `normalized/`, `annotation/` and `results/`.

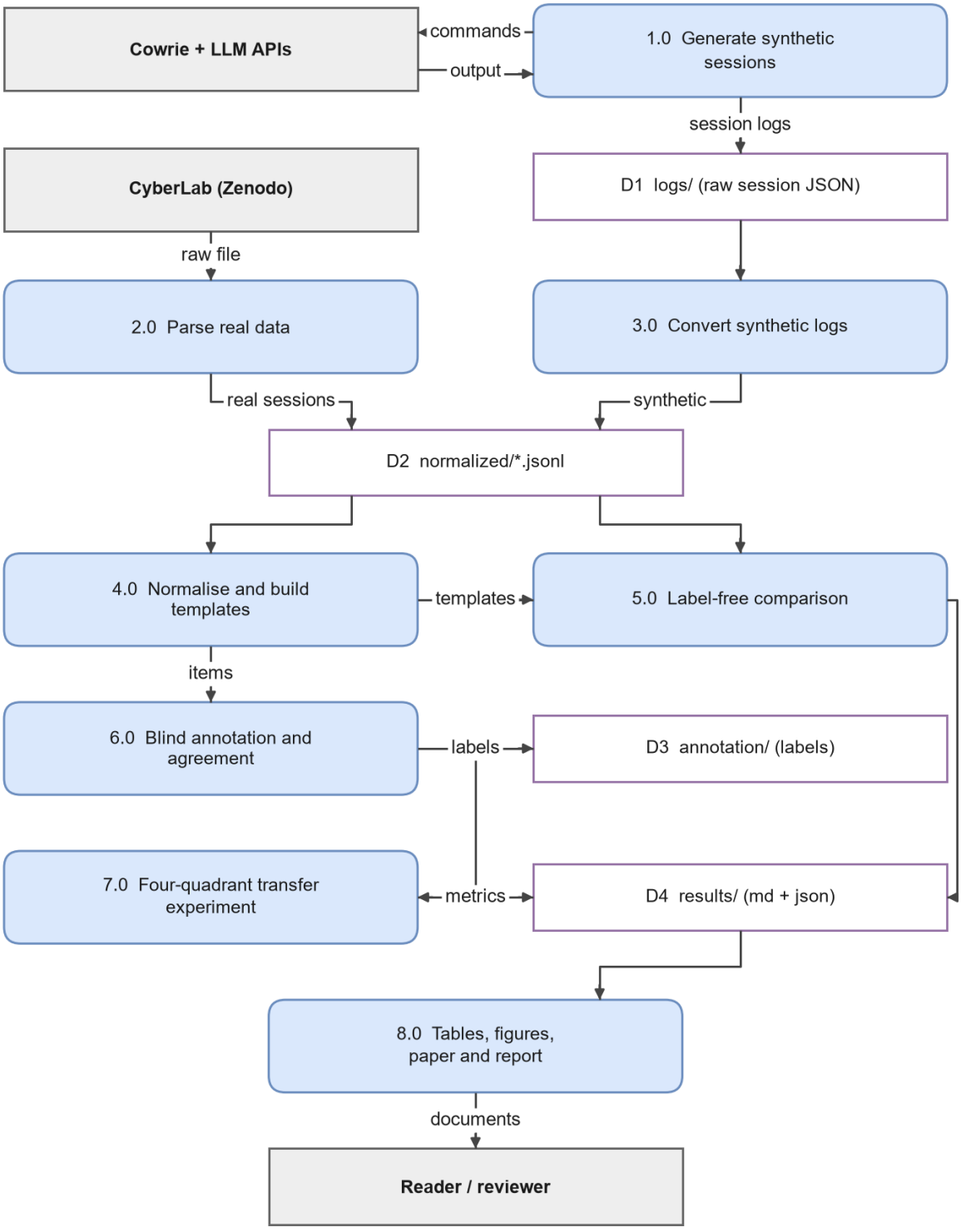


Figure 2.2: Data-flow diagram, level 1.

2.3 Components

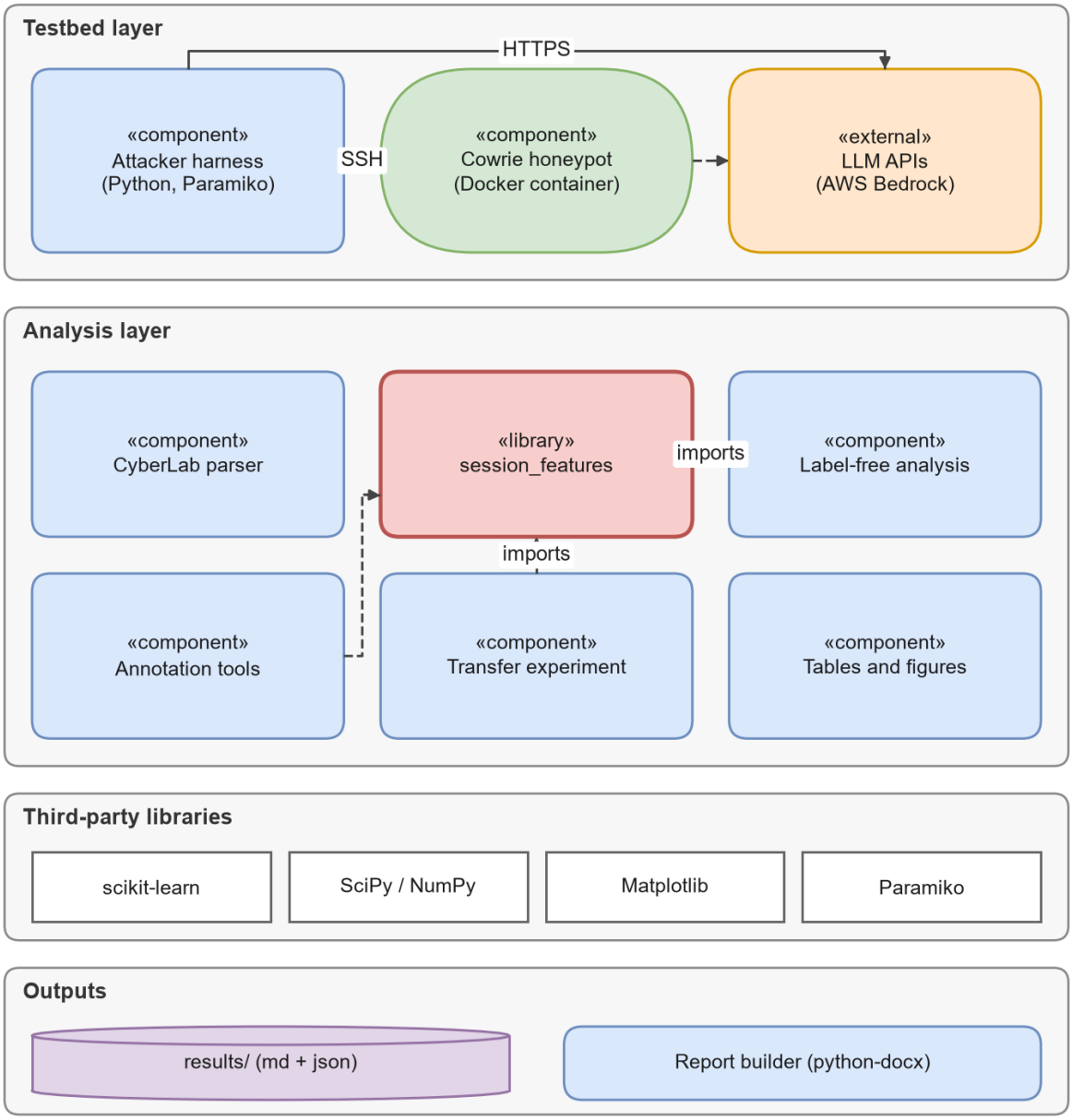


Figure 2.3: Component diagram.

The most important design rule is visible in **Figure 2.**3: every analysis component imports the same `session_features` library, so synthetic and real sessions are always normalised, split and tagged in exactly the same way.

2.4 Deployment

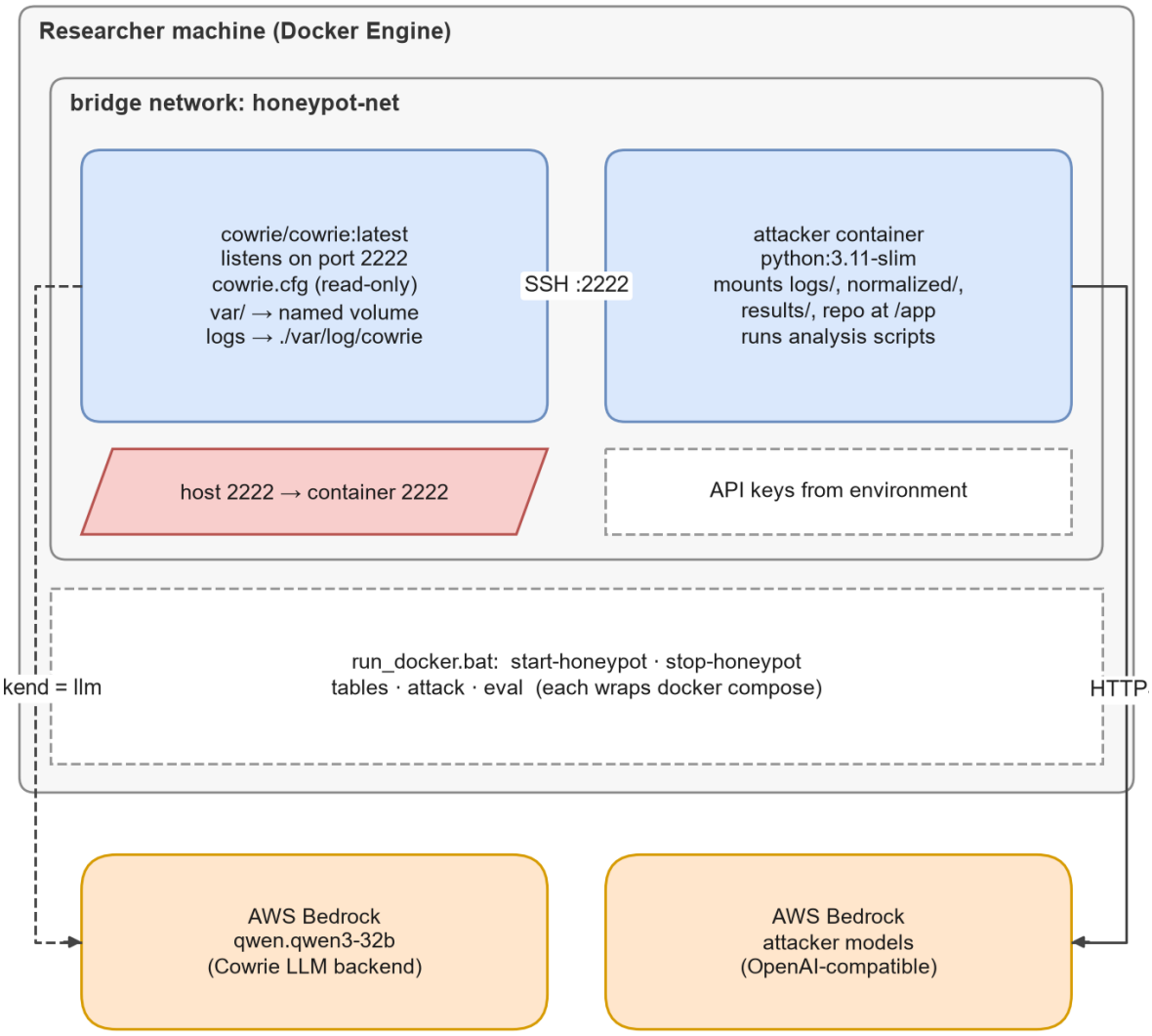


Figure 2.4: Docker deployment of the testbed.

The honeypot and the analysis code run as two Docker containers on a private bridge network (**Figure 2.**4). Only the analysis container needs internet access, to reach the model API; the Cowrie container reaches its own model API only when the LLM backend is switched on.

2.5 Repository layout

Table 2.1: Repository layout.

Path	Contents
`better_ai_attacker.py`	Attacker harness used for the study
`cowrie_etc/cowrie.cfg`	Honeypot configuration (backend switch)
`docker-compose.yml`, `Dockerfile.attacker`, `run_docker.bat`	Containers and Windows helper
`logs/`	120 raw synthetic session logs
`parse_cyberlab.py`	Parser for the CyberLab export
`session_features.py`	Shared normalisation, verbs, tactic tags, templates
`normalized/`	Normalised synthetic and real corpora
`domain_shift.py`	Label-free comparison
`make_annotation_sheet.py`, `agreement.py`, `annotation/`	Annotation kit and labels
`train_eval_classifier.py`	Four-quadrant transfer experiment
`generate_paper_tables.py`, `make_figures.py`	Paper tables and figures
`results/`, `figures/`	All reported numbers and paper figures
`paper/`	Springer LNNS paper (Overleaf project)
`report/`	Project report: Markdown, figures, draw.io sources, build scripts
`docs/`	This documentation

3. Installation and Setup

3.1 Requirements

Table 3.1: Software requirements.

Requirement	Version	Needed for
Python	3.11	Everything
pip packages	`requirements.txt`	paramiko, boto3, anthropic, openai, requests, scikit-learn, scipy, numpy, pandas
Git	any	Getting the code
Docker Desktop	any recent	Running the honeypot (optional for analysis only)
Microsoft Word	365 / 2019+	Building the report (optional)
matplotlib, python-docx	recent	Building the report figures and the Word file (optional)

3.2 Steps

```
git clone https://github.com/parthkansal823/Honeypot_RedTeaming-paper.git
cd Honeypot_RedTeaming-paper
python -m venv venv
venv\Scripts\Activate.ps1
pip install -r requirements.txt
```

3.3 The real dataset

Download `cyberlab_2019-11-15.json.gz` from Zenodo (DOI 10.5281/zenodo.3687527) into the project root and extract it so that `cyberlab_2019-11-15.json/cowrie_2019-11-15.json` exists. The file is about 600 MB uncompressed; the parser streams it, so a normal laptop is enough.

3.4 API keys

Keys are read from environment variables and are never stored in files:

Table 3.2: Environment variables.

Variable	Used by
`BEDROCK_API_KEY`	Attacker models and the Qwen3-32B honeypot backend through the OpenAI-compatible Bedrock endpoint
`OPENAI_API_KEY`, `ANTHROPIC_API_KEY`, `GEMINI_API_KEY`	Optional alternative providers

You only need keys to generate *new* synthetic sessions. The 120 sessions used in the study are already in `logs/`.

3.5 Checking the installation

```
python -c "import paramiko, sklearn, scipy, numpy; print('ok')"
python domain_shift.py      # should print the label-free tables
```

4. Running the Pipeline

All commands run from the repository root. Each step writes its results to files, so the steps can be run one at a time and inspected in between.

4.1 Overview

Table 4.1: Pipeline steps.

#	Step	Command	Output
1	Parse the real data	`python parse_cyberlab.py`	`normalized/real_organic.jsonl`
2	Label-free comparison	`python domain_shift.py`	`results/domain_shift.md`, `.json`
3	Blind annotation sheets	`python make_annotation_sheet.py`	`annotation/` sheets and key
4	Agreement and final labels	`python agreement.py`	`annotation/labels_final.csv`, `results/annotation_agreement.md`
5	Transfer experiment	`python train_eval_classifier.py …` (see 4.6)	`results/table3_transfer.md`, `.json`
6	Attacker tables	`python generate_paper_tables.py`	`results/tables_1_2.md`, `.json`
7	Paper figures	`python make_figures.py`	`figures/*.png`, `*.pdf`
8	Report	see `report/README.md`	`report/Project_Report.docx`, `.pdf`

4.2 Optional: the live honeypot

```
.\run_docker.bat start-honeypot     # Cowrie on localhost:2222
python check.py                     # prints the SSH banner a few times
```

To switch Cowrie to the LLM backend, set `backend = llm` in `cowrie_etc/cowrie.cfg`, fill in the `[llm]` section and restart the container. New synthetic batches are run with the harness against this local honeypot only (see the project report, Section 3.7, for its settings).

4.3 Step 1: parse the real data

```
python parse_cyberlab.py --in cyberlab_2019-11-15.json\cowrie_2019-11-15.json --out
normalized\real_organic.jsonl
```

The summary should report 76,785 sessions and 1,672 with commands. If it reports zero sessions, the wrong parser was used: `cowrie_session_parser.py` reads Cowrie's native JSON-lines logs, not the CyberLab export.

4.4 Step 2: label-free comparison

```
python domain_shift.py
```

Prints and saves corpus structure, tactic shares, verb JSD with bootstrap confidence intervals, Jaccard overlap and the classifier two-sample test, plus per-cell distances.

4.5 Steps 3 and 4: annotation

`make_annotation_sheet.py` writes two identical blind sheets, a shared item list and a hidden key. Each annotator fills the `label` column of their sheet using `annotation/RUBRIC.md`, without seeing the key or the other sheet. Disagreements are resolved in `annotation/adjudicated.csv`. `agreement.py` then computes Cohen's κ, writes the disagreements file and produces `labels_final.csv`, broadcasting each real template's label to all its sessions.

4.6 Step 5: transfer experiment

```
python train_eval_classifier.py --synthetic normalized\all_synthetic_120.jsonl `
  --real normalized\real_organic.jsonl `
  --real-labels annotation\labels_final.csv --synthetic-labels annotation\labels_final.csv
`
  --label-source llm-dual
```

Without the label options the script falls back to keyword-rule labels. Those labels share keywords with the features, so that run is only a plumbing check; its output is kept separately in `results/rule_labels/`.

4.7 Steps 6 to 8: tables, figures, report

```
python generate_paper_tables.py
python make_figures.py
```

The report has its own build described in `report/README.md`.

5. Data Formats

5.1 Overview

Figure 5.1 shows how the records relate. Raw synthetic logs are converted into the same normalised session shape as the real data; each real template and each synthetic session becomes one annotation item; final labels are broadcast back to sessions.

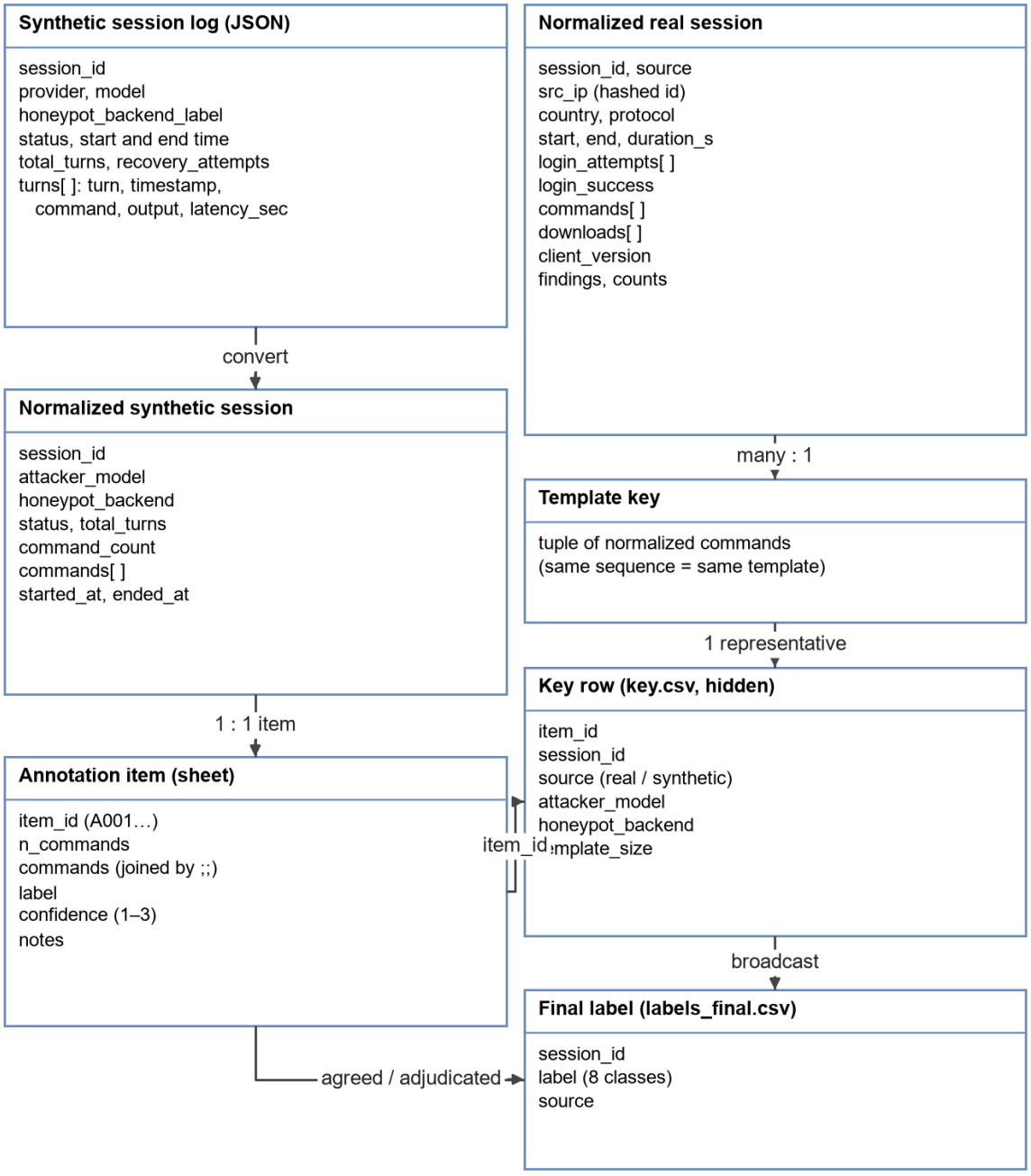


Figure 5.1: Data model.

5.2 Raw synthetic session log (`logs/**/<session_id>.json`)

Table 5.1: Raw synthetic session log fields.

Field	Type	Meaning
session_id	string	provider-model-index-random hex
attacker_type	string	always `synthetic_llm`
provider, model	string	how the attacker model was reached
honeypot_backend_label	string	`static` or `dynamic_qwen.qwen3-32b`
status	string	how the session ended (completed, max_turns_reached, timed_out, attacker_refused, llm_error, connection_lost, honeypot_unresponsive, connection_failed)
started_at, ended_at	ISO time	UTC timestamps
total_turns	integer	number of recorded turns
turns	array	`{turn, timestamp, command, output, latency_sec, note}`

5.3 Normalised session (`normalized/*.jsonl`, one JSON object per line)

Table 5.2: Normalised session fields.

Field	Type	Meaning
session_id	string	unique id
commands	list of strings	commands in order
attacker_model, honeypot_backend, status	string	synthetic sessions only
source	string	`cyberlab_2019-11-15` for real sessions
src_ip	string	hashed source identifier (real only)
country, protocol	string	geolocation code and `ssh` / `telnet` (real only)
start, end, duration_s	time / number	session timing (real only)
login_attempts	list	`{username, password, success}` (real only)
downloads	list	`{url, shasum, outfile}` (real only)
command_count	integer	number of commands

5.4 Annotation files (`annotation/`)

Table 5.3: Annotation files.

File	Columns
`annotation_sheet.csv`	item_id, commands
`sheet_annotator1.csv`, `sheet_annotator2.csv`	item_id, n_commands, commands, label, confidence, notes
`key.csv` (hidden while labelling)	item_id, session_id, source, attacker_model, honeypot_backend, template_size
`adjudicated.csv`	item_id, label, rationale
`disagreements.csv`	item_id, label_annotator1, label_annotator2, notes1, notes2, commands, label_final
`labels_final.csv`	session_id, label, source
`RUBRIC.md`	the eight labels, examples and the tie-break order

5.5 Result files (`results/`)

Every analysis writes a Markdown file for reading and a JSON file with the same numbers for other scripts:

Table 5.4: Result files.

File	Contents
`domain_shift.md`, `.json`	corpus structure, tactics, verbs, JSD, C2ST, per-cell distances
`annotation_agreement.md`	κ by subset, confusion matrix, final label counts
`table3_transfer.md`, `.json`	four-quadrant results, label distribution, per-class F1, predictions
`tables_1_2.md`, `.json`	attacker behaviour per cell and session outcomes
`sensitivity_dota/`	transfer results with the dota template relabelled
`rule_labels/`	keyword-rule plumbing check (not a result)

6. Script Reference

6.1 Analysis scripts

Table 6.1: parse_cyberlab.py options.

Option	Default	Meaning
`--in`	`cyberlab_2019-11-15.json/cowrie_2019-11-15.json`	CyberLab daily file (one JSON array)
`--out`	`normalized/real_organic.jsonl`	Output, one session per line

Table 6.2: domain_shift.py options.

Option	Default	Meaning
`--synthetic`	`normalized/all_synthetic_120.jsonl`	Synthetic corpus
`--real`	`normalized/real_organic.jsonl`	Real corpus
`--out-dir`	`results`	Where `domain_shift.md` and `.json` are written

Fixed settings inside the script: seed 42, 500 bootstrap samples, 5 × 5 repeated stratified cross-validation for the classifier two-sample test.

Table 6.3: make_annotation_sheet.py options.

Option	Default	Meaning
`--real`	`normalized/real_organic.jsonl`	Real corpus (one item per distinct command sequence)
`--synthetic`	`normalized/all_synthetic_120.jsonl`	Synthetic corpus (one item per session)
`--out`	`annotation`	Output folder
`--seed`	42	Shuffle seed

Table 6.4: agreement.py options.

Option	Default	Meaning
`--dir`	`annotation`	Folder with sheets, key and adjudication
`--a1`, `--a2`	`sheet_annotator1.csv`, `sheet_annotator2.csv`	Annotator sheets
`--adj`	`adjudicated.csv`	Adjudicated labels for disagreements
`--key`	`key.csv`	Hidden key
`--real`	`normalized/real_organic.jsonl`	Used to broadcast template labels
`--md`	`results/annotation_agreement.md`	Agreement report

Table 6.5: train_eval_classifier.py options.

Option	Default	Meaning
`--synthetic`, `--real`	required	Corpora
`--real-labels`, `--synthetic-labels`	none	Label CSVs (`session_id,label`); without them, keyword-rule labels are used (plumbing check only)
`--label-source`	`human`	Name printed in the results header, for example `llm-dual`
`--out-dir`	`results`	Output folder

Fixed settings: TF-IDF with at most 2,000 tokens plus three numeric features; random forest with 300 trees, balanced class weights, seed 42; GroupKFold(5) by template for real data; macro-F1 over classes present in the test set.

6.2 Shared library: session_features.py

Table 6.6: Main functions of session_features.py.

Function	What it does
`normalize_cmd(cmd)`	Cuts leaked chat tokens, lower-cases, masks URLs, IPs, quoted strings, long hex, random tokens, numbers and random busybox tags
`segments(cmd)`	Splits a normalised command on `;`, `|`, `&&`, `||`, newline, `$(` and backtick
`verb(segment)`	First word of a segment, ignoring `VAR=value` prefixes and paths
`session_verbs(session)`	All verbs of a session
`session_tactics(session)`	Set of descriptive tactic names matched by the regular expressions
`template_key(session)`	Tuple of normalised commands; equal keys mean the same behaviour template
`has_chat_leak(cmd)`, `strip_chat_leak(cmd)`	Detect and remove leaked chat-format tokens

6.3 Testbed scripts

Table 6.7: Testbed scripts.

Script	Purpose
`better_ai_attacker.py`	Harness used for the study; settings are described in the report (Section 3.7)
`run_docker.bat`	`start-honeypot`, `stop-honeypot`, `tables`, `attack`, `eval` shortcuts around Docker Compose
`check.py`	Reads the honeypot's SSH banner a few times
`probe.py`	Sends a short test prompt to each candidate model endpoint
`validate_logs.py`	Scans old logs for marker-leak corruption

6.4 Report scripts (`report/src/`)

Table 6.8: Report build scripts.

Script	Purpose
`compute_stats.py`	Descriptive statistics for the report
`make_diagrams.py` (+ `diagram_lib.py`, `diagrams_a4.py`, `diagrams_software.py`)	All flowcharts and UML diagrams as `.drawio` and `.png`
`make_charts.py`	Data charts and console pictures
`make_appendix_md.py`	Data appendices generated from project files
`build_report.py`	Markdown → Word (`--docs` builds this documentation)
`finalize_word.ps1`	Native equations, field refresh and PDF export through Word

7. Results Summary

All numbers below are copied from `results/`. The project report discusses them in detail.

7.1 Label-free comparison

Table 7.1: Label-free comparison of synthetic and real sessions.

Measure	Synthetic	Real
Sessions	120	1,672 (22 templates)
Share of top-5 templates	4.2%	96.2%
Mean characters per command	43.0	53.3
Commands chained or piped	19.4%	40.3%
Verb JSD (synthetic vs real)	0.700 (95% CI 0.688–0.717)	
Verb vocabulary Jaccard	0.285	
Classifier two-sample AUC (commands / verbs only)	1.000 / 0.996	

7.2 Labels

Cohen's κ between the two blind annotators was 0.898 over 184 items (172 agreed, 12 adjudicated). Most synthetic sessions are credential access (87 of 120); most real sessions are resource hijacking (1,040) or persistence (443).

7.3 Transfer

Table 7.2: Four-quadrant transfer results.

Quadrant	Macro-F1	Accuracy	Majority F1
Q1 synthetic → synthetic	0.493	0.758	0.168
Q2 real → real	0.983	0.992	0.001
Q3 synthetic → real	0.205	0.022	0.000
Q3-dedup (one per template)	0.071	0.045	0.000
Q4 real + synthetic → real	0.981	0.992	0.001

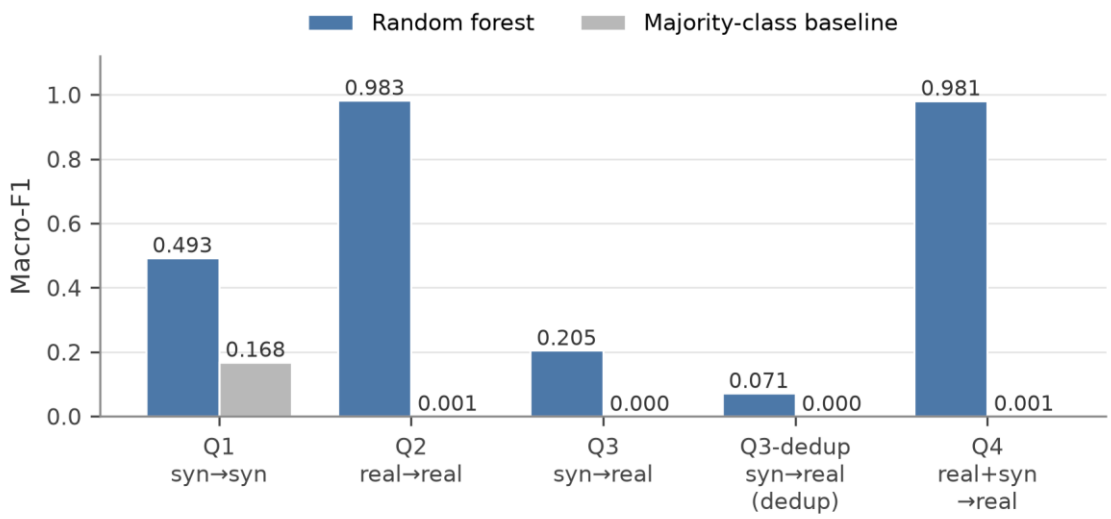


Figure 7.1: Macro-F1 per quadrant.

7.4 Robustness

Relabelling the dota template (and the other resource-hijacking templates) as malware drop leaves Q3 near zero (0.021) while Q2 and Q4 reach 0.933 and 0.965. The keyword-rule run collapses Q3 to 0.000 but is only a plumbing check.

7.5 Other findings

- GPT-OSS-120B leaked chat-format control tokens and reasoning into the shell in 18 of 40 sessions (11 static, 7 dynamic).
- The LLM-backed honeypot raised the share of credential-hunting commands for every attacker and moved only Gemma-3-4B closer to real traffic (JSD 0.798 → 0.746).
- A parser written for native Cowrie logs reads zero sessions from the CyberLab export without raising an error.

8. Troubleshooting and FAQ

8.1 Common problems

Table 8.1: Problems and fixes.

Symptom	Cause and fix
The parser reports 0 sessions	Wrong parser or the file is still gzip-compressed. Use `parse_cyberlab.py` on the extracted file
The honeypot stops answering	A heavy command froze Cowrie's single-threaded reactor; restart the container (`run_docker.bat stop-honeypot` then `start-honeypot`)
Honeypot replies are cut short	The LLM backend is slow; increase the harness's response wait
Commands contain English sentences	Chat-format leakage; make sure commands are cut at the first `<|`
Scores look too good	Check that real folds are grouped by template and that labels did not come from keyword rules
`agreement.py` says the sheets are blank	Fill the `label` column of both annotator sheets first
The Word report shows "Update field"	Open it in Word, press Ctrl+A then F9
`finalize_word.ps1` cannot save	Close the report in Word (and any preview pane), then run it again

8.2 Frequently asked questions

Why not just use keyword rules as labels? The classifier's features contain the same keywords, so it would only re-learn the rules. Rule labels are kept only as a plumbing check.

Why are the real labels "broadcast" from templates? Many real sessions are byte-for-byte the same script after masking. Labelling one representative per distinct sequence and copying the label to its template is consistent and saves effort; it also means the effective real sample size is 22 templates, which the results always report.

Why macro-F1 over present classes only? A class that the model predicts but that never appears in the test set would add a zero to the average and make the score look worse than it is.

Are the LLM labels as good as human labels? They agree with each other very well ($\kappa = 0.898$) and were checked by the authors, but they come from one provider and are not an independent human audit. The report lists this as the main threat to validity.

Can I use the harness against other systems? No. It is designed and documented for use against your own honeypot only.

Where is the editable version of each diagram? In `report/diagrams/*.drawio`; open them in draw.io or diagrams.net.